Computer Science & IT



Compiler Design

Syntax Analysis (Parser)

Lecture No. 09



By- Vishal Sir

Recap of Previous Lecture



Topic — Parsing using LL(1) parsing table

Topic — Conditions to check LL(1) grammar

Topic — Introduction to Bottom-up Parsers

Slide

Topics to be Covered



- Topic: LR(0) Parser
- Topic: SLR(1) Parser

Slide



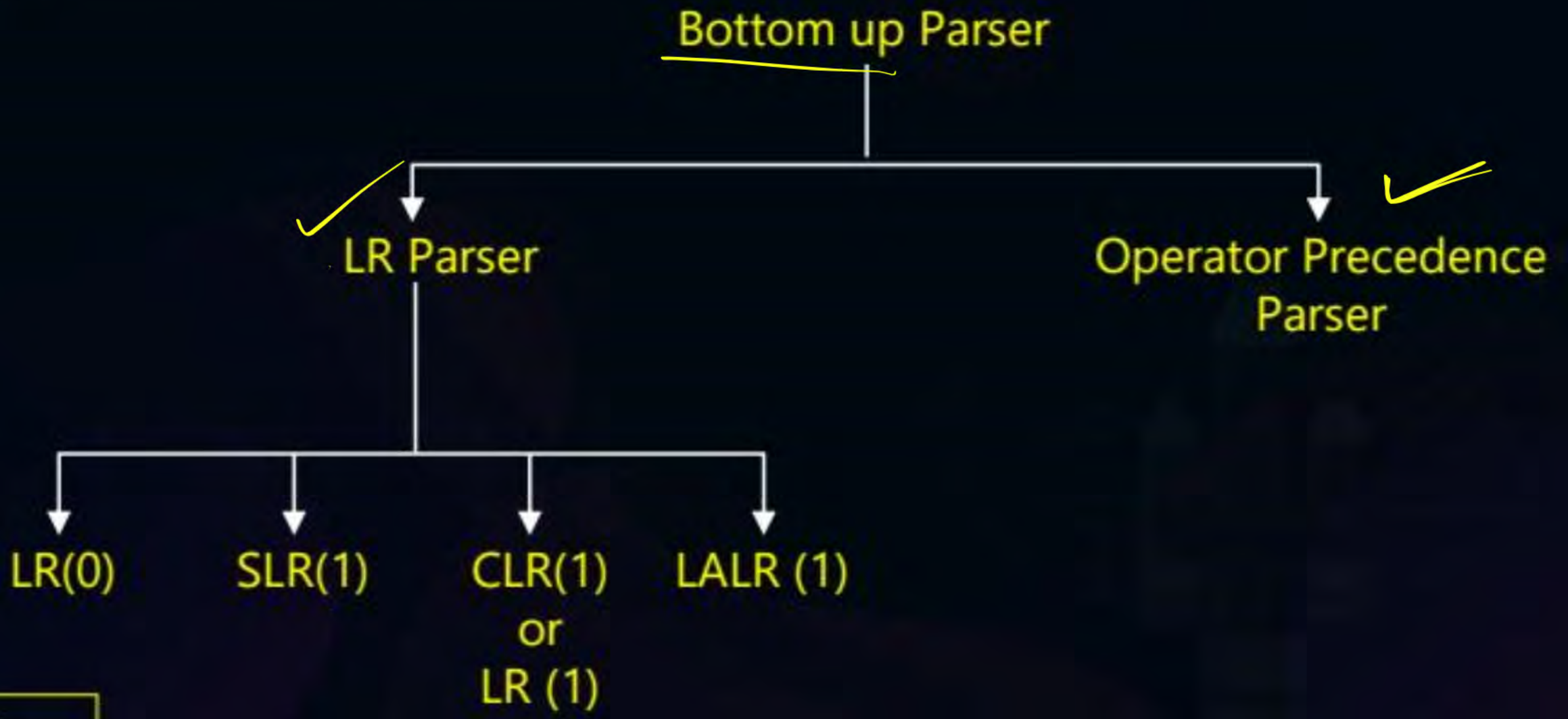
Topic : Bottom-up Parsers

- Bottom Up Parser starts constructing the parse tree starting from the given input string and proceeds until the starting symbol of the grammar is obtained.
- If a parser uses bottom up approach for parsing the string, then it follows reverse of rightmost derivation in the construction of parse tree.



Topic : Types of Bottom-up Parsers

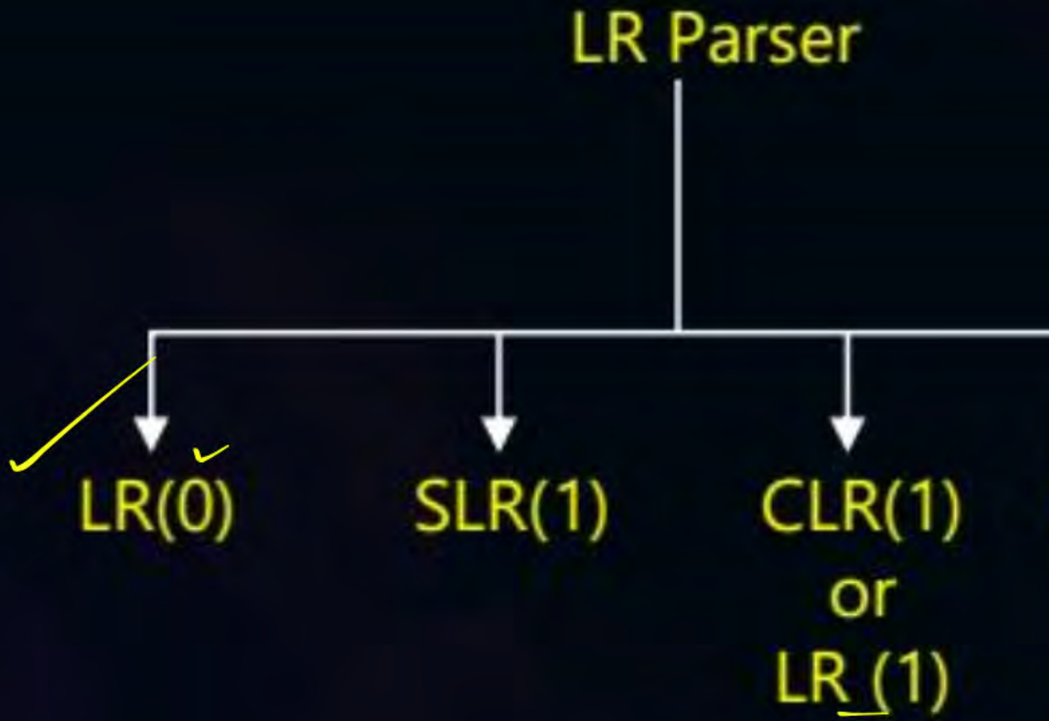




Slide



Topic : Types of LR Parsers



Slide



Topic : Handle

Handle is a sub-string in the right sentential form {i.e. sentential form w.r.t. right most derivation} that matches with the Complete right hand side part of a production of the grammar and Can be reduced into the non-terminal On the left hand side of that production.

Slide



Topic : Handle

Consider the following grammar

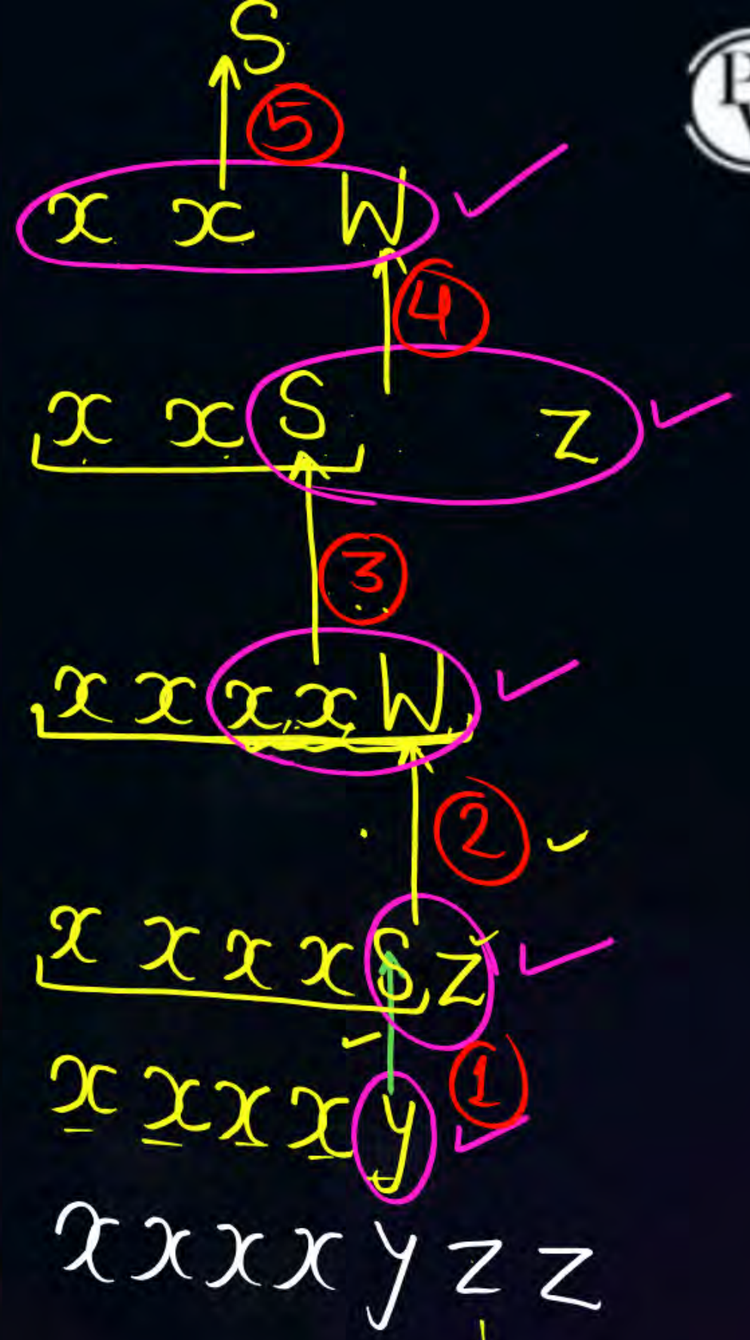
$S \rightarrow xxW$

$S \rightarrow y$

$W \rightarrow Sz$

How many numbers of handles are detected by bottom-up parser for the string = x x x x y z z

$S \Rightarrow xxW \Rightarrow xxSz \Rightarrow xxxxWz \Rightarrow xxxxSzz \Rightarrow xxxxyzz$



Slide

* Number of handles detected by bottom-up parser = Number of Reduce actions Performed by bottom-up Parser



Topic : Augmented Grammar

Augmented grammar is the same grammar With an extra production {known as augmented production} of type $S' \longrightarrow S$

Augmented production

Where,
S is the starting symbol of original grammar,
and S' is a newly added non-terminal

Slide

* Augmented production is added into the grammar to represent Successful parsing.

Whenever we reduce S into S' it represents Successful parsing

eg:

Grammar

$S \rightarrow AA$

$A \rightarrow aA$

$A \rightarrow b$



Augmented grammar

$S' \rightarrow S]$ ← Augmented Production

$S \rightarrow AA$

$A \rightarrow aA$

$A \rightarrow b$



Topic : LR(0) item

* LR(0) item is a production of the grammar having a DOT (.) mark somewhere on the RHS part of the production

eg: $A \rightarrow \alpha . \beta$ {α, β may be any thing}

Slide



$A \rightarrow XYZ$ is the Production

Topic : LR(0) item

eg i) $A \rightarrow .XYZ$ { We have seen nothing, and we are ready to see the string derivable from XYZ }

ii) $A \rightarrow X.YZ$ { We have seen the string derivable from X, and we are ready to see the string derivable from YZ }

iii) $A \rightarrow XY.Z$ { We have seen the string derivable from XY, And we are ready to see the string derivable from Z }

iv) $A \rightarrow XYZ.$ { We have seen the string derivable from XYZ. And there is nothing to be seen i.e. We are ready to reduce w.r.t $A \rightarrow XYZ$ }

Slide

$A \rightarrow .\epsilon \equiv A \rightarrow \epsilon.$

We have seen nothing and, there is nothing to see. $\equiv$ We have seen nothing, and there is nothing to see

If we have $A \rightarrow .\epsilon$ then we can simply convert it into $A \rightarrow \epsilon.$ {i.e. we are ready to reduce w.r.t $A \rightarrow \epsilon$}

Slide

Closure of LR(0) item :-

→ Obtain closure of LR(0) item I { i.e. Closure(I) }

→ Step-1 :- Include Item I itself into closure(I)

→ Step2 :- If these exists any item of the form $A \rightarrow \alpha.B\beta$ in closure(I) { $\alpha, \beta \in (V+T)^*$ and B is a non-terminal } such that $B \rightarrow \gamma$ is a production in the grammar and $B \rightarrow .\gamma$ is not already in closure(I), then add $B \rightarrow .\gamma$ in closure(I).

Slide

↳ Step 3:- Repeat Step-2 for every newly added Production (if possible)

eg:-

Grammar

$S \rightarrow AA$

$A \rightarrow aA$

$A \rightarrow b$

$\Rightarrow$ Augmented grammar

$S' \rightarrow S$

$S \rightarrow AA$

$A \rightarrow aA$

$A \rightarrow b$

* Consider the following LR(0) item

$S' \rightarrow \cdot S$

Obtain Closure $(S' \rightarrow \cdot S)$

Closure $(S' \rightarrow \cdot S) =$

$S' \rightarrow \cdot S$

$S \rightarrow \cdot AA$

$A \rightarrow \cdot aA$

$A \rightarrow \cdot b$

* Construction of DFA of LR(0) items w.r.t. given grammar

* ① → obtain the augmented grammar,
let, $S' \rightarrow S$ is the augmented production

② Construct States of the DFA :-

① Construct initial State

② Start a New State

③ Complete the Construction of newly started State

④ Repeat step② & step③ until no more State Can be started

① Construction of initial state of DFA of LR(0) items.

If $S' \rightarrow S$ is the augmented production, then Closure$(S' \rightarrow .S)$ will define the items in the initial state of the DFA of LR(0) items

② Starting a new state in the DFA of LR(0) items

* If $A \longrightarrow \alpha . X \beta$ is an item in the state 'i' of the DFA of LR(0) items, { where $\alpha, \beta \in (V+T)^*$ and X is a symbol of length one { terminal / Non-terminal } }

then locate all the items in state 'i' such that, X immediately follows the Dot(.) mark, and start a new state 'j' by moving the Dot(.) mark after X in all those items

③ Complete the Construction of newly started State:-

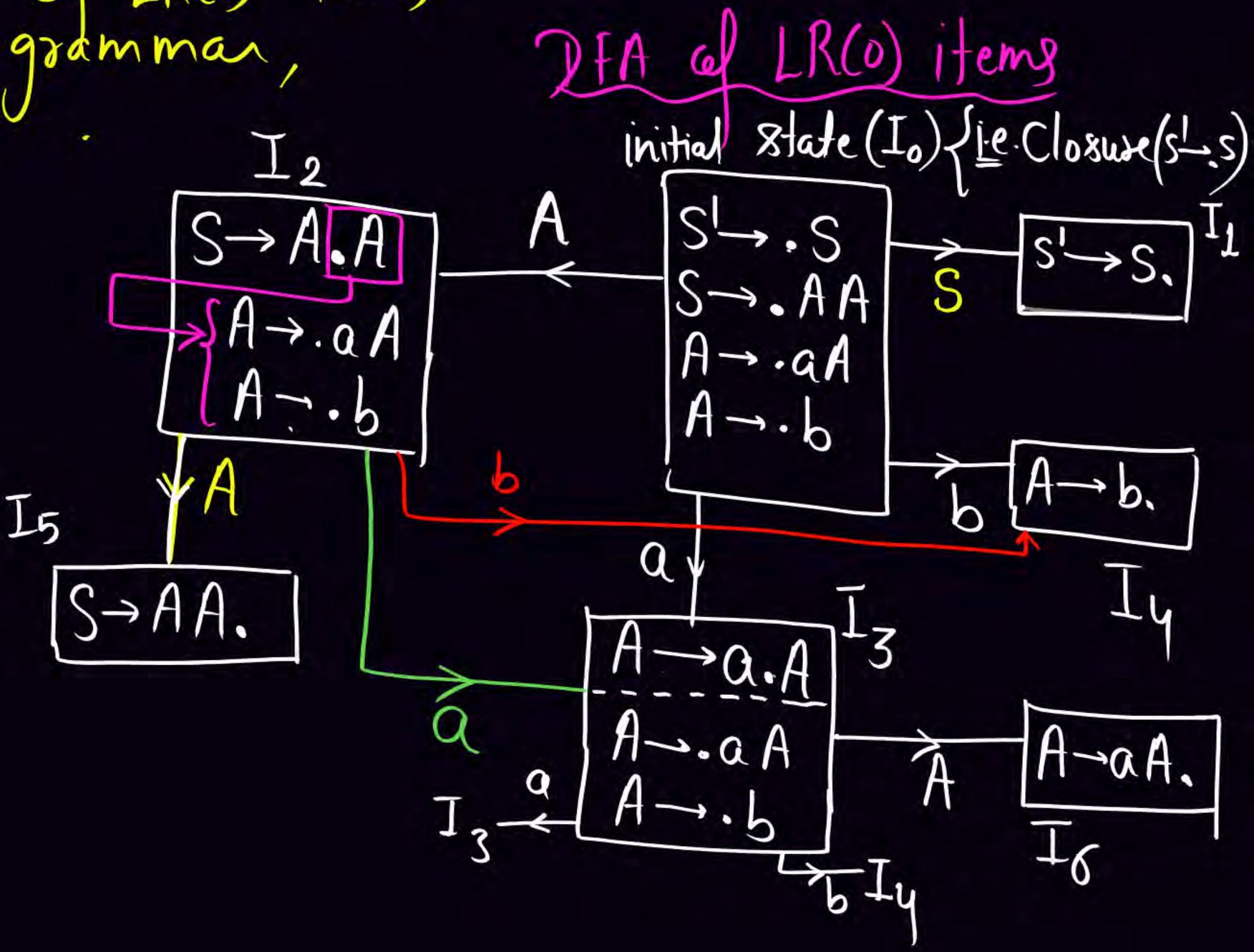
i.e obtain the closure of the LR(0) items with which the state is started.

* In newly started state 'j' if there exists an item of the form $A \longrightarrow \alpha.B\beta$ and $B \longrightarrow \gamma$ is a production of the gramman and if $B \longrightarrow .\gamma$ is not already present in state 'j' then add $B \longrightarrow .\gamma$ in state 'j'.

And Repeat this process for every newly added item in state 'j'.

④ Repeat ② & ③ until no more states can be started

* Construct the DFA of LR(0) items for the following grammar,



* Construct the DFA of LR(0) items for the following grammar,

G_2:
1. $S \rightarrow AA$
2. $A \rightarrow aA$
3. $A \rightarrow b$

aab$

⇓

Augmented grammar

$S' \rightarrow S$
$S \rightarrow AA$
$A \rightarrow aA$
$A \rightarrow b$

DFA of LR(0) items

initial state (I_0) { i.e. Closure($S' \rightarrow .S$)

I_0:
$S' \rightarrow .S$
$S \rightarrow .AA$
$A \rightarrow .aA$
$A \rightarrow .b$

S → I_1: $S' \rightarrow S.$

A → I_2: $S \rightarrow A.A$, $A \rightarrow .aA$, $A \rightarrow .b$

A → I_5: $S \rightarrow AA.$ (1st production)

a → I_3: $A \rightarrow a.A$, $A \rightarrow .aA$, $A \rightarrow .b$

A → I_6: $A \rightarrow aA.$ (2nd production)

b → $A \rightarrow b.$ (3rd production)

LR(0) Parser :-

① obtain augmented grammar

② Obtain DFA of LR(0) items

③ Convert DFA of LR(0) items into LR(0) Parsing table

④ If no entry of the Action part of LR(0) parsing table Contains more than one option then Corresponding grammar is LR(0) grammar

⑤ If any entry of the action part Contain more than one option then not LR(0) grammar

✡ Conversion of DFA of LR(0) item into LR(0) Parsing table.

Note:- LR parsing table w.r.t. all LR parsers Consists of two parts
① Action Part ② Goto Part

→ Action Part of LR Parsing table will Consists of 4 types of entries

1. Shift
2. Reduce
3. Accept
4. Error

* Action part entries wrt. LR(0) parsing table

① **Shift** :- If state 'i' of the DFA of LR(0) items contains an item of the form

$A \longrightarrow \alpha . a \beta$ { where 'a' is any terminal } (must be terminal for shift)

and there is a transition out of state 'i' to state 'j' on terminal 'a', then

Update Action [i, a] = S_j { from state 'i' on terminal 'a' shift to state 'j' }

(i : state, a : terminal)

② Reduce :-

∴ If State 'i' of the DFA Contains an item of the form

$A \longrightarrow \alpha .$, {i.e ready to reduce w.r.t $A \longrightarrow \alpha$}

Where $A \longrightarrow \alpha$ is the jth production of the grammar, then update R_j {i.e. Reduce w.r.t. Production 'j'} in all the entries of Action part Corresponding to State 'i'

Note { No look-ahead is used to place reduce entry in the parsing table } LR(0)

③ Accept :- If state 'i' of the DFA Contains an item of the form

$S' \longrightarrow S.$, {where $S' \longrightarrow S$ is augmented Production}

then update

Action [i, $] = Accept

④ Error :-

Set all other entries of Action part as Error {i.e. blank entries in Action Part correspond to Error}

* Goto Part of LR parsing table :-

* Generally Goto part is same for all parsing table.

* Goto entries with respect to LR(0) Parser

* If these exists any item in state 'i' of the form

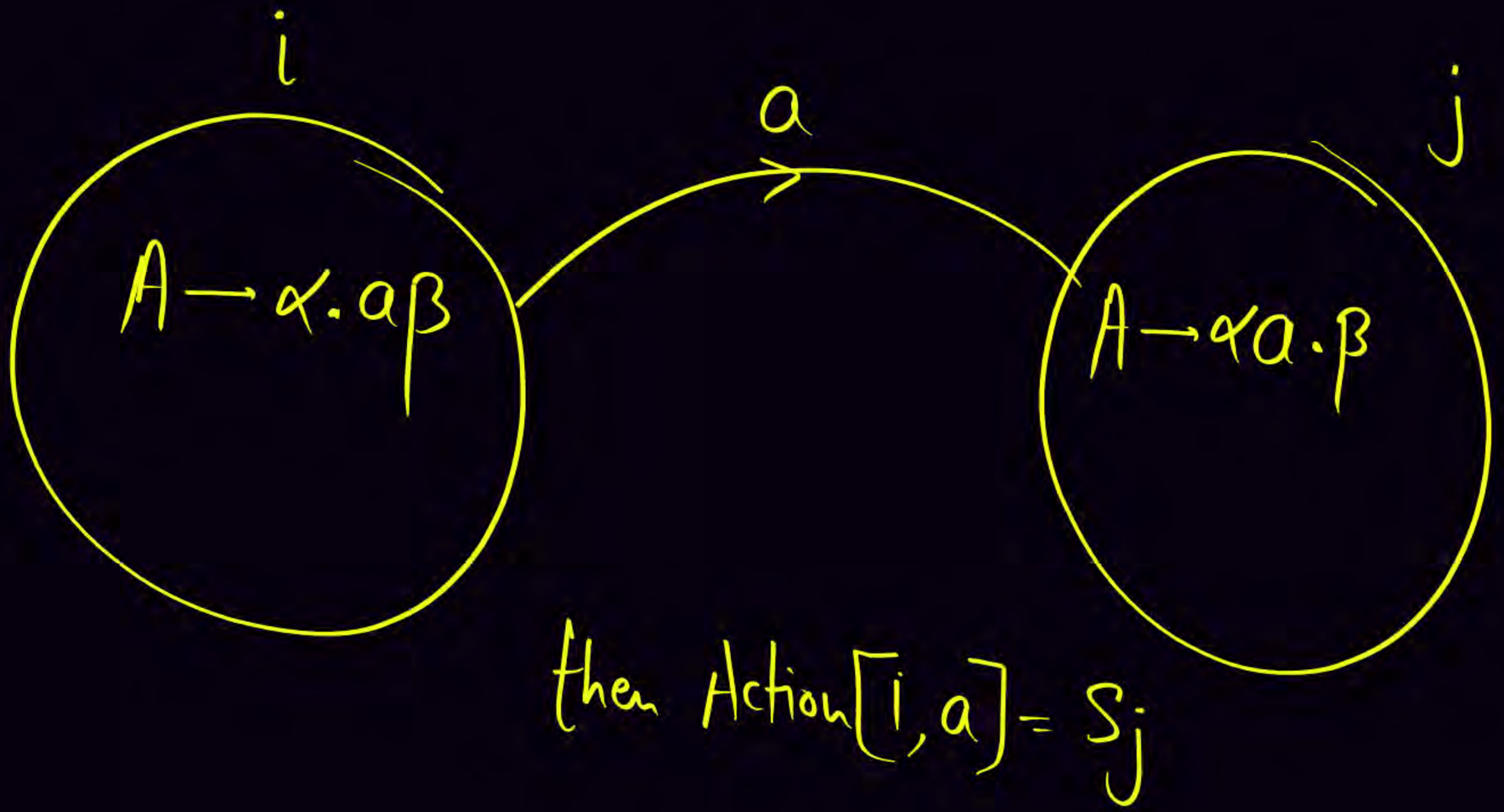
$$A \longrightarrow \alpha . B \beta$$ { where B is the non-terminal }

(must be non-terminal)

and these is a transition out of state 'i' to state 'j' on non-terminal B, then update

Goto [i, B] = j

(i: State, B: Non-terminal, j: New State)



then Action[i, a] = s_j


Action
Goto
a
b
$
S
A
0
S3
S4
1
2
1
Accept
2
S3
S4
5
3
S3
S4
6
4
R3
R3
R3
5
R1
R1
R1
6
R2
R2
R2
States of the DFA



2 mins Summary

Topic LR(0) Parser

Topic SLR(1) Parser

Slide

THANK - YOU

Slide